הרצאה 2 – שפות תכנות 2027

מרצה: הדר פרנקל

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

איפה אנחנו בקורס? נמשיך לעסוק היום בפרולוג. שבוע שעבר, ראינו כיצד כותבים תוכנית בפרולוג. השבוע – נמשיך לעסוק ברשימות, נדבר כיצד פרולוג מוצא את התשובות שלו, ובהמשך ההרצאה – נדבר על תוכניות גדולות יותר ועל אריתמטיקה וכו'. שנצא לדרך?

המשך רשימות

תזכורת. רשימה ריקה מסומנת ע"י [] ורשימה מורכבת מראש – איבר בודד H וזנב T שהוא רשימה בפני עצמו.

- _ בתחילה: משתנה אנונימי. משמעותו "לא אכפת לנו". למשל [T|_] המשמעות: לא אכפת לנו מיהו ראש הרשימה.

```
?- append(X,Y,[1,2,3]).
X = [],
Y = [1, 2, 3] ;
X = [1],
Y = [2, 3] ;
X = [1, 2],
Y = [3] ;
X = [1, 2, 3],
Y = [] ;
false.
```

הפרדיקט append: הפרדיקט לוקח שתי רשימות Y,X ומחזיר משתנה Z שהוא שרשור שתי הרשימות X וY. לדוגמה: אם נשים שני משתנים X וY נקבל את כל האפשרויות שיצרו את הרשימה [1,2,3]:

כיצד מממשים את הפרדיקט?

- כלל בסיס: אם משרשרים רשימה ריקה לרשימה – מקבלים את אותה רשימה.
- צעד: אם יש לנו רשימה שמורכבת מראש וזנב [Head|TailA] ועוד B – אזי הרשימה המשורשרת היא למעשה הרשימה [Head|TailC] **אם** (וזה התנאי, הגדרת הכלל): כאשר משרשרים את TailA עם B מקבלים את TailC. בעצם זהו הצעד הרקורסיבי שמוריד לי איבר (ה"ראש") H.

```
append([], B, B).
append([Head|TailA], B, [Head|TailC]) :-
  append(TailA, B, TailC).
```

הפרדיקט Select: הפרדיקט מקבל X וY כאשר X הוא איבר בתוך הרשימה Y, והוא "מחזיר" רשימה Z שהיא הרשימה Y לאחר שמורידים את האיבר X.

לדוגמה:

```
?- select(2,[1,2,3],Z).
Z = [1, 3] ;
false.
```

הפרדיקט גמיש, כלומר ניתן לשים משתנים בשאילתא.

עוד פרדיקטים:

Predicate	Description
member(X,Y)	Provable if the list **Y** contains the element **X**.
append(X,Y,Z)	Provable if the list **Z** is the list **X** followed by the list **Y**.
select(X,Y,Z)	Provable if the list **Y** contains the element **X**, and **Z** is the same as **Y** but with one instance of **X** removed.
nth0(X,Y,Z)	Provable if **X** is an integer, **Y** is a list, and **Z** is the **X**th element of **Y**, counting from 0.
nth1(X,Y,Z)	Same as **nth0**, but counting from 1.
length(X,Y)	Provable if **X** is a list of length **Y**.
reverse(X,Y)	Provable if **Y** is the list **X** in reverse order.

- רובם גמישים, למשל Append. שאילתות יכולות להיות עם משתנים. היחיד שלא – Reverse.

<u>הפרדיקט Reverse</u>: הפרדיקט לוקח את הרשימה $X = [X_1, \ldots, X_n]$ והופך אותה לתוך המשתנה $Y = [X_n, \ldots, X1]$. כיצד מממשים אותו?

```
reverse([],[]).
reverse([Head|Tail],X) :-
  reverse(Tail,Y),
  append(Y,[Head],X).
```

- בסיס: רשימה ריקה – נשארת ריקה.
- צעד: אם יש לנו ראש וזנב, נרצה להפוך את הזנב לתוך Y ולהוסיף את הראש בסוף הרשימה ההפוכה Y.

נשים לב כי זהו מימוש לא טוב. Append היא עצמה פונקציה רקורסיבית לכן נקבל שבכל קריאה יש לנו קריאה לפונקציה שעולה זמן לינארי. לכן סה"כ זמן הריצה הוא $O(n^2)$ וזה לא יעיל. גם סיבוכיות הזיכרון לא טובה במיוחד.

<u>דרך נוספת למימוש של הפרדיקט. יותר מסובך:</u>

```
reverse_tr(Xs, Ys) :- rev_loop(Xs, [], Ys).
rev_loop([], Acc, Acc).
rev_loop([X|Xs], Acc, Ys) :- rev_loop(Xs, [X|Acc], Ys).
```

- משתמשים בaccumulator שמטרתו לשמור לנו נתונים שחישבנו בעבר על מנת להקל בזמן הריצה.
- יש לנו פונקצייה עזר rev_loop:
  - מקרה הבסיס שלה – אם X הוא ריק אזי שני המשתנים האחרים יהיו שווים לבסוף – כי שרשור רשימה ריקה לרשימה חייב להשאיר את הרשימה כפי שהיא. הAccumulator נכנס לתוך YS.
  - הרקורסיה: אם אין רשימה ריקה, אזי יש איזשהו X ראש וזנב Xs. אנחנו מעבירים את ראש הרשימה לתוך Acc שזוכר לנו איברים והזנב הופך לרשימה המקורית. כך אנחנו ממשיכים עד שהרשימה המקורית מתרוקנת – ואז דואגים כי בזכות הכלל בו שני המשתנים האחרונים שווים – כי הפלט: הרשימה ההפוכה, הוא בדיוק הפלט שקיבלנו בAcc.
- סיבוכיות הזמן כאן היא לינארית, וסיבוכיות המקום לינארית גם כן. נתבונן בדוגמת הרצה:

```
          to do     Acc
rev_loop([1,2,3], [],      Ys)
rev_loop([2,3],   [1],     Ys)
rev_loop([3],     [2,1],   Ys)
rev_loop([],      [3,2,1], Ys)  Ys=[3,2,1]
```

<u>שימוש נכון בפרדיקטים על רשימות</u>

ראשית, נרצה להתבונן בפרדיקטים שלא מסתיימים לעולם. נחזור למימוש הנאיבי של reverse.

```
nrev([],[]).
nrev([Head|Tail],X) :- nrev(Tail,Y), append(Y,[Head],X).
?- nrev(X,[1,2,3,4]).
X = [4, 3, 2, 1] ;
```

ביצענו שאילתא – למעשה אמרנו לו: ההפוך של הרשימה שלנו הוא [1,2,3,4] - תמצא לנו את הרשימה המקורית. בפעם הראשונה הוא אכן יצליח, אם נמשיך ממנו עם נקודה פסיק עוד השמות אפשריות הוא יתקע ולא יחזור לעולם. מדוע? הוא מחפש רשימה שתהיה הפוכה לרשימה שנתונה לו, אבל אחרי שהוא מצא את הרשימה הזו הוא ימשיך לחפש ולא יסיים לעולם. **באופן כללי** נתקל במקרים כאלו כאשר יש לנו פרדיקטים מסובכים ושמנו משתנה בשאילתא על הקלט. מדוע זה קורה? הוא ימשיך לחפש להפוך רשימות באורכים יותר גדולים ובכלל לא יגיע לappend. הוא יישאר בnrev הראשון.

- המימוש של Prolog לReverse הוא המימוש עם Accumulator, והוא בודק גם את אורך הרשימה. ו**מקרה זה לא קורה במימוש של פרולוג.**

דוגמה נוספת. Sort לוקח רשימה וממיין אותה. אם נרצה לבצע את הדבר ההפוך – לתת רשימה שאם נמיין אותה נקבל [1,2,3,4] הוא ייתן לנו שגיאה. יש כאן הרבה אפשרויות לכאורה – אבל לכאורה זה נראה מספר סופי. האמנם? לא! יש אינסוף רשימות שהמיון שלהן יכול להחזיר את זה – מדוע? כי <u>המימוש של פרולוג לSort מוחק כפילויות</u>. לכן הרשימה שבה יש מיליון פעמים 1, מיליון פעמים 2, 67 פעמים 3 ופעם אחת 4 – גם היא מחזירה את הרשימה הנ"ל.

```
?- sort([2,3,1,4],X).
X = [1, 2, 3, 4].

?- sort(X,[1,2,3,4]).
ERROR: Arguments are not sufficiently instantiated
```

<u>משתנה אנונימי</u>

בכל פעם שנראה _ הוא יכול להתקשר למשהו אחר, גם אם רואים כמה באותה שאילתא. זהו משתנה אנונימי.

נתבונן לדוגמה בפרדיקט tailOf: הוא מקבל רשימה X שאיננה ריקה, וY שהוא הזנב (שיכול להיות ריק) של הרשימה X. הוא מחזיר 1 אם זה אכן נכון.

```
tailof([_|A],A).
```

<u>מימוש אפשרי נוסף</u> – באופן הבא, אבל אנחנו נקבל אזהרה. למה אזהרה? כי כלל לא השתמשנו במשתנה Head, לכן עדיף להוסיף משתנה אנונימי עם _.

```
tailof([Head|A],A).
?- consult(tailof).
Warning: tailof.pl:1:
Warning:    Singleton variables: [Head]
```

מסקנה. לא כדאי להתעלם מהערות.

<u>שלילה ועקרון העולם הסגור</u>

שלילה בפרולוג אנחנו נכתוב ע"י \+G, יש ממשקים שבהם ניתן לכתוב not(G) : ואז הוא יצליח – אם G לא יצליח.
למשל:

```
?- member(1,[1,2,3]).
true .

?- \+ member(4,[1,2,3]).
true.

?- \+ member(1,[1,2,3]).
false.
```

- בדוגמה הראשונה, אכן 1 נמצא ברשימה.
- בדוגמה השנייה, אכן 4 לא נמצא ברשימה – לכן מחזיר אמת.
- בדוגמה האחרונה – 1 כן נמצא ברשימה, ולכן זה לא נכון שהוא לא ברשימה. לכן זה מחזיר שקר.

אמנם פרולוג היא שפת תכנות לוגית, אך אין זה אומר שזה שקול לx¬: זה מחזיר אמת אם הפסוק המקורי נפסל. אם המקורי רץ אינסוף – גם השלילה ירוץ אינסוף. חשוב לשים לב לזה.

- הוא בודק בלבד, ולא יכול לבצע קשירות למשתנים.

דוגמה. נתבונן במסד הנתונים המשפחתי משמאל:

נרצה להגדיר את הכלל ש*X* ו*Y* הם אחים. כיצד?

```
parent(kim,holly).
parent(margaret,kim).
parent(margaret,kent).
parent(esther,margaret).
parent(herbert,margaret).
parent(herbert,jean).
```

```
sibling(X,Y) :-
  parent(P,X),
  parent(P,Y),
  \+ X = Y.
```

- X,Y הם אחים כאשר ישנו הורה P שהוא הורה משותף של שניהם, **וגם** X **איננו** Y. זה חשוב – אחרת כל אחד יכול להיות אח של עצמו.

```
?- sibling2(kim, kent).
true.

?- sibling2(X, Y).
false.
```

הסדר שאנחנו בודקים הוא משנה. <u>אם היינו מגדירים את הכלל קודם כל בבדיקה כי X שונה מY – זה היה עובד לא טוב.</u> **מדוע?** כי X וY יכולים לקבל מה שהם רוצים – וNOT אומר לא רק שלילה לוגית אלא שלילה פרוצדורלית, X איננו Y המשמעות "לא קיימים ערכים X כך שהם שונים מY" – ומשפט זה הוא לא נכון, ממש ישנם כאלו שאפשריים. ולכן כל השאילתא נכשלת. לכן אם אנחנו קוראים עם זוג משתנים – לא טוב ולא נכון להגדיר את השאילתא כך שX שונה מY נבדק קודם.

<u>Closed world assumption:</u> הנחת העולם הסגור. בפרולוג אנחנו מניחים שכל מה שאין לנו בעולם – תמיד מוגדר להיות שקר, מה שלא נתון כעובדה – הוא שקר גמור.

Unification

כעת נדבר על יוניפיקציה בצורה פורמלית. עד כה, דיברנו על **איך** כותבים תוכניות פרולוג – חוץ מאריתמטיקה עליה נרחיב בהמשך. עכשיו, אנחנו רוצים לדבר על זוג מנגנונים מאוד חשובים בפרולוג: יוניפיקציה וBacktracking: איך פרולוג מחפש פתרונות?

על מנת להגדיר יוניפיקציה (התאמת המשתנים בשאילתא) נגדיר Subtitution (החלפה) להיות פונקציה שלכל משתנה מתאימה term.

$$\sigma = \{\mathbf{X} \rightarrow \mathbf{a},\ \mathbf{Y} \rightarrow \mathbf{f(a,b)}\}$$

F הוא פרדיקט, כך גם G. X וY הם משתנים.

פרולוג מבצע יוניפיקציה לשני terms נסמנם T_1, T_2 אם קיימת substitution σ כך שיהפוך אותם לזהים. כלומר $\sigma(T_1) = \sigma(\text{T_2})$

לדוגמה:

- **a** and **b** do not unify
- **f(X,b)** and **f(a,Y)** unify: a unifier is {**X**→**a**, **Y**→**b**}
- **f(X,b)** and **g(X,b)** do not unify
- **a(X,X,b)** and **a(b,X,X)** unify: a unifier is {**X**→**b**}
- **a(X,X,b)** and **a(c,X,X)** do not unify
- **a(X,f)** and **a(X,f)** do unify: a unifier is {}

- המקרה הראשון ברור. כך גם השני.
- במקרה השלישי – לא ניתן לבצע התאמה כי f שונה מg. גם אם מימשנו אותם זהה.
- המקרה החמישי לא יתכן – הוא דורש כי X=b=c וזה לא יתכן.

<u>כמובן שיתכנו מספר פונקציות החלפה אפשריות.</u> למשל עבור parent(X,Y), parent(X,fred) ישנם כמה אפשרויות לX, ולכן כמה אפשרויות לפונקציית החלפה.

דוגמה חשובה.

נגדיר חוק להלן. ונבצע שאילתא להלן. נשים לב כי פרולוג אכן מבצע יונפיקציה כי X הוא פיצה. אבל, זה לא תואם לכלל ללא הfood כמובן – זה לא אותו דבר!

Rule:
likes(john, food(pizza)).
Query:
?- likes(john, food(X)).
Prolog unifies by setting:
X = pizza
But this would **not unify** with
likes(john, pizza).
because food(pizza) $\neq$ pizza.

אז כיצד מתבצעת בפועל היוניפיקציה?

```
parent(john, mary).
parent(mary, alice).
grandparent(X, Y) :-
    parent(X, Z), parent(Z, Y).
```

- נניח ונבצע שאילתא grandparent(john, Who) אזי נקבל who = Alice. כיצד זה עובד?
- אנחנו מתאימים את המטרה grandparent לראש (X, Y)

ע"י הפונקציה {X- > john, Y- > who} .

- אנחנו מתאמים את parent(john, Z) שהוא המטרה שלנו עם העובדה הראשונה Z- > MARY במסד הנתונים. ואז המטרה השניה parent(Mary, Who) מתאימה לאליס. וסיימנו.

Backtracking

כאשר פרולוג מקבל איזושהי שאילתא, הוא מתחיל ממנה ומנסה להוכיח אותה. הניסיון להוכיח מתחיל בשאילתא שקיבלנו – הולך לעובדות והחוקים ומנסה לחפש. החיפוש הוא מלמעלה למטה, משמאל לימין. אנחנו מנסים להשתמש ביוניפיקציה – אם היא מצליחה אנחנו מקשרים את המשתנים ושומרים אותם, ואם לא נבצע באק-טראקינג ונחפש התאמה אחרת.

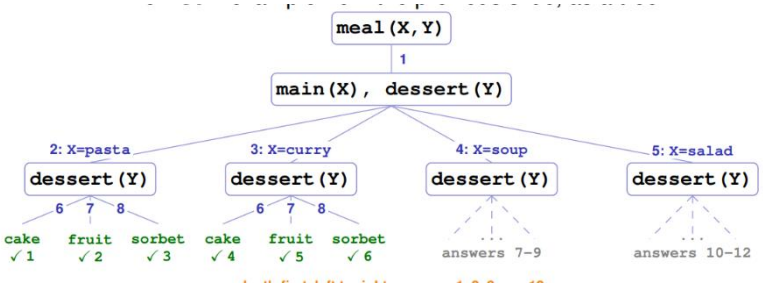
```
1  meal(X,Y) :-
       main(X), dessert(Y).
2  main(pasta).
3  main(curry).
4  main(soup).
5  main(salad).
6  dessert(cake).
7  dessert(fruit).
8  dessert(sorbet).
```

נתבונן במסד הנתונים עם הכלל הבא.

ונתבונן בשאילתא Meal(X, Y).

למעשה הגדרנו ארוחה של X וY אם X הוא ארוחה עיקרית וY הוא קינוח.
אנחנו למעשה שואלים – האם קיימת ארוחה?

- הוא מתחיל בחיפוש מלמעלה למטה. הוא מתחיל בלקשור את *X* לפסטה, ואז הוא מחפש התאמות ל*Y*. הראשון שיראה הוא עוגה – ולכן יתאים את פסטה ועוגה. לאחר מכן הוא ישחרר את *Y* בלבד, וימצא לאיקס את ההתאמה הבאה, והוא יתאים את פסטה גם לפירות וסורבה. ואז הוא ישחרר את *X* - יעבור לcurry וכן הלאה. כלומר הוא עובר לפי הסדר בתוך הכלל: משמאל לימין, ומלמעלה למטה בעובדות.



בהתאמה ניתן ליצור את העץ שמתאים לבאק-טראקינג. עלה בעץ הוא תשובה: התאמה לשני המשתנים שיוצרים ארוחה. הוא למעשה סורק את העץ הזה בצורת Depth-first .

נאמר כי Node הוא נקודת בחירה, אם בחרנו בו משהו. נשים לב כי אנחנו מתקדמים בעץ – לא בודקים את אותם הדברים שוב ושוב.

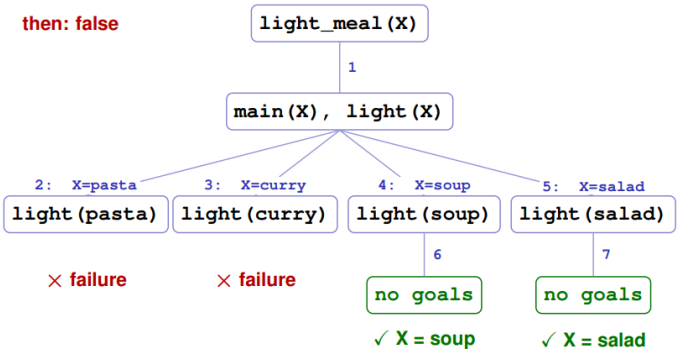
אם כן, זו דוגמה פשוטה יחסית – כל האפשרויות עבדו. נתבונן בדוגמה מורכבת יותר:

```
1. light_meal(X) :-
    main(X), light(X).
2. main(pasta).
3. main(curry).
4. main(soup).
5. main(salad).
6. light(soup).
7. light(salad).

?- light_meal(X).
```

כן הגדרנו ארוחה קלה, כוללת מנה עיקרית ומנה קלה. נניח שאילתא light_meal(X)

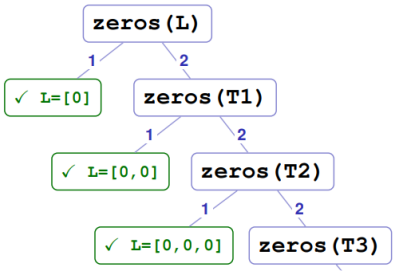
כיצד נראה במקרה זה עץ החיפוש?




```
1. zeros([0]).
2. zeros([0|T]) :- zeros(T).
```

דוגמה מורכבת יותר. האם הרשימה מורכבת מאפסים בלבד?

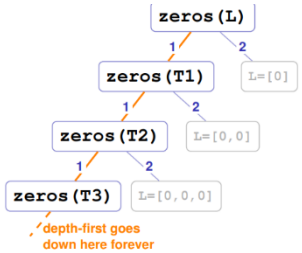
נניח כי אנחנו במקרה הזה, וכיצד יראה עץ החיפוש על שאילתא של מציאת רשימה? יש אינסוף רשימות כאלו שמלאות באפסים. הוא לעולם לא ישיב לנו שקר כי הוא תמיד יבנה את הרשימה הבאה – וימשיך לבנות לנצח בצורה הזו:




```
1. zeros([0|T]) :- zeros(T).
2. zeros([0]).
```

עכשיו, נניח כי כתבנו את אותו כלל בדרך שונה, הפוך:

כאן לא נקבל תשובה לעולם, הוא לעולם לא יצור רשימה ולכן הוא רק יתקדם ברקורסיה. לכן כאן אם היינו שואלים האם קיימת רשימה – היינו מקבלים לא, בניגוד לדוגמה הקודמת שם היינו מקבלים שקיימת רשימה.



Cut

לפעמים נרצה לעצור את פרולוג מלרוץ עוד אחרי שקיבלנו תשובה אחת – או חלילה לרוץ לנצח. זה נקרא cut והוא מסומן ע"י !. המשמעות היא: אם מצאת binding שעובדות – תתחייב אליהן. אל תנסה לבצע להן התאמות נוספות. הוא למעשה חותך לנו ענפים בעץ החיפוש שלא בדקנו – אם מצאנו כבר התאמה.

```
light_meal(X) :-
    main(X), light(X).

?- light_meal(X).
X = soup ;
X = salad.
```

```
first_light(X) :-
    main(X), light(X), !.

?- first_light(X).
X = soup.
```

דוגמה ראשונה. אם נריץ את השאילתא עם ! בסיום – אנחנו נקבל רק תשובה אחת. מדוע? הוא מתחייב להשמה המקורית של מרק, והוא לא ממשיך לחפש את סלט.

הערה. דברים שמופיע לאחר ה! – להם לא נתחייב ליוניפיקציה שלהם.

דוגמה שנייה.

Without the cut

```
p :- member(X,[a,b,c]),
     write(X).
p :- write(d).
```

```
?- p.
a
true ;
b
true ;
c
true ;
d
true.
```

With the cut

```
p :- member(X,[a,b,c]),
     write(X), !.
p :- write(d).
```

```
?- p.
a
true.
```

נניח תוכנית שבודקת אם איבר X הוא חלק מרשימה נתונה, ואם כן היא מדפיסה אותו. בסיום היא מדפיסה את d (איבר קבוע).

משמאל, קוד רגיל – ללא !. זה אכן מדפיס את שלושת האפשרויות ובסיום מדפיס את d.

מימין – נשים לב שהוא מדפיס את a בלבד כמו שצפינו. אבל ה! טוען – אל תבדוק שום אפשרויות נוספות לספק את p בהמשך, זהו אותו כלל ולכן הוא לא ממשיך ומדפיס את d. אם למשל היינו שמים את השורה עם הדפסת d למעלה – זה היה מדפיס d,a וזהו.

עד כה ראינו רק מתי הcut הורס לנו, אז מתי כן טוב להשתמש בזה? לדוגמה:

```
has_light_main :-
    main(X), light(X), !.

?- has_light_main.
true.
```

זה חוק שלא ראינו עד כה – זה חוק שבודק קיום. הוא בודק האם בכלל קיימת ארוחה קלה – שהמנה העיקרית היא גם קלה. לחוק שכזה **אין משתנים**. החוק הנ"ל ניתן להוכחה אם ורק אם קיים משתנה X שהוא גם ארוחה וגם קל. נשים לב שהוא לא יחזיר לנו את המשתנה! X הוא פנימי לתוך החוק, ולחוק עצמו אין משתנים ומטרתו רק לשאול האם קיימת. הוא לא רוצה לקבל 10 פעמים את התשובה TRUE – מספיק וקיימת לו השמה אחת.

אריתמטיקה

```
?- 1+2*3 = 7.
false.
```

כפי שכבר ראינו, בדוגמה הזו נקבל שקר. מדוע? מבחינת פרולוג הסימונים הללו הם סה"כ פרדיקטים והוא מפרש אותם בתוך פרדיקטים ולא כסימנים מתמטיים.

```
?- X is 1+2*3.
X = 7.
```

על מנת לבדוק אם אכן הביטוי שווה לשבע, נכתוב:

בדיוק בשביל זה משתמשים בis – פרדיקט, X is E.

```
?- Y = X+2, X = 1.      % no evaluation: Y is a term
Y = 1+2, X = 1.

?- Y is X+2, X = 1.     % X unbound when is runs
ERROR: Arguments are not sufficiently instantiated

?- X = 1, Y is X+2.     % same goals, other order
X = 1, Y = 3.

?- 7 is X+2.            % is does not solve equations
ERROR: Arguments are not sufficiently instantiated
```

1. דוגמה ראשונה: כאן מבצעים יונפיקציה וY מקבל את 1+2.
2. דוגמה שנייה: עם is זה כבר לא יעבוד. הוא רוצה לחשב את הערך של Y לפי X, אבל אי אפשר לעשות זאת כי X מוגדר רק לאחר מכן.
3. דוגמה שלישית: כעת זה כתוב בסדר הנכון, הוא יודע את הסדר של X ולכן יכול לחשב את Y.
4. דוגמה רביעית: שוב שגיאה, הוא לא יכול לבצע את החישוב כך. עלינו לדאוג כי המשתנה X יהיה מצד שמאל לis, כלומר שהמידע שיהיה לו כיצד לחשב את המשתנה חייב להיות מימין לis. צד ימין חייב להיות ידוע לגמרי! קבוע. ושוב, **הסדר משנה.**

```
?- X is 1/2.
X = 0.5.

?- X is 2/1.
X = 2.

?- X is 2.0/1.0.
X = 2.0.

?- X is 7 // 2.
X = 3.
```

יש לנו שני סוגי מספרים. Integer וReals. מספר דוגמאות:

- // הוא חילוק בשלמים – מקבלים את הערך השלם ללא שארית. זה בInteger.
- לProlog אין בעיה שעושים overloading: מבחינתו אם פעם אחת X הוא שלם או ממשי, אז לא אכפת לו: אין לו הגדרה מראש אם הוא שלם או ממשי. **עם זאת,** אם נשאל אותו האם 2.0 = 2 נקבל שזה נכשל! כי מבחינתו מדובר בשני terms שונים. לכן, גם אין יונפיקציה בין 2 לו2.0.

השוואות:

`X < Y`	less than
`X > Y`	greater than
`X =< Y`	less or equal
`X >= Y`	greater or equal
`X =:= Y`	equal
`X =\= Y`	not equal

```
?- 1+2 >= 1+1.
true.
```

- קודם כל מבוצעת השמה לצדדים – ואז מתבצעת ההשוואה.

דוגמה: אם יש גם משתנים ויש השמה להם – זה מחזיר אותה.

```
?- X is 1-3, Y is 0-2,
   X =:= Y.
X = Y, Y = -2.
```

<u>סוגים של שוויון:</u>

Goal	What it does	3 ? 1+2	1+2 ? 3
X = Y	unify, no evaluation	fails	fails
X is Y	evaluate Y, unify with X	succeeds	fails
X =:= Y	evaluate both, compare numbers	succeeds	succeeds

- שווה רגיל = הוא של יוניפיקציה.
- Is מבצע אבולואציה לצד ימין (קבוע) ומבצע לזה יוניפיקציה עם X.
- =:= זה כבר שוויון רגיל, משווה מספרים.

- עם שוויון מסוג =:= אזי כן מתקיים 2.0=:=2 כלומר מוחזר 1. למרות שזה לא אותו term.

דוגמה. פונקציה שמחשבת סכום של רשימה. נעיר כי ניתן לערבב שלמים ולא שלמים ברשימה.

```
sum([],0).
sum([H|T],S) :-
  sum(T,S1),
  S is H + S1.
```

כיצד מממשים? מקרה הבסיס הוא רשימה ריקה, הסכום שלה הוא אפס. אחרת – אזי ניתן לייצגה ע"י ראש וזנב. הסכום שלה – הוא הסכום של הזנב, ועוד האיבר הראשון. וכך ברקורסיה.

דוגמה. חישוב עצרת. נבנה פונקציה שמקבלת X מס' ומחזירה את תוצאת !X. במקרה הבסיס, אם X שווה ממש ל1 אזי נעצור. נחזיר 1. אחרת, אנחנו מניחים כי X גדול מאחד. אזי, נגדיר משתנה חדש להיות X-1 ונריץ עליו ברדוקציה, את התוצאה נשמור בתוך NF. ואז הפלט הוא כמובן X*NF.

```
factorial(X,1) :-
  X =:= 1, !.
factorial(X,F) :-
  X > 1,
  NewX is X - 1,
  factorial(NewX,NF),
  F is X * NF.
```

<u>פעולות שאינן פרדיקטים שמאפשרות לקרוא/לכתוב ולשנות את מסד הנתונים</u>

- Write(T) הוא פרדיקט שמדפיסה את T.
- Read(X) הוא פרדיקט שקורא X term. הוא יחכה שנכתוב משהו – מה שנכתוב יכנס למשתנה X.
- nl זה ירידת שורה.

```
p :-
  append(X,Y,[1,2]),
  write(X), write(' '),
  write(Y), nl,
  X = Y.
```

```
?- p.
[] [1,2]
[1] [2]
[1,2] []
false.
```

- דוגמה נחמדה: בודק אם קיימים X וY שווים ששרשורם נותן את הרשימה לעיל. נשים לב כי ללא ההדפסות היינו מקבלים רק false. באמצעות ההדפסות למסך, אנחנו רואים את הבאק-טראקינג שהוא עשה ורואים את כל האפשרויות.

יש בידינו אפשרות לשנות את התוכנית עצמה תוך כדי ריצה. פרולוג לא כל כך מחבב דבר זה כמובן, ולכן אם נרצה לאפשר לשנות פרדיקט תוך כדי ריצה עלינו להצהיר עליהם בתוך פרדיקטים דינאמיים כך:

```
:- dynamic parent/2.
parent(kim,holly).
```

כעת ניתן לשנות עובדה זו בזמן ריצה. כיצד? נוסיף עובדה חדשה באמצעות assert.

```
parent(joe,mary).          false.
assert(parent(joe,mary)).  true.
```

- ניתן לבצע assert אם ורק אם אפשרנו זאת עם דינאמיק. הוא יוסיף את זה בסוף מסד הנתונים. אם נעשה asserta הוא יוסיף בתחילת מסד הנתונים.

```
?- retract(parent(joe,mary)). true.
```

- נוכל גם להוריד עובדה בזמן הריצה, הtrue שנקבל לאחר הרצת הפקודה אומר שהוא הצליח להוריד את העובדה ממסד הנתונים. כאן נשתמש בretract.

ניתן גם לבצע משהו גרוע יותר – ולהוריד מישהו שהוא הבן של קים. הוא יוריד רק את הראשון שיראה. הוא יחזיר את מי שהוא הוריד. אם נעשה נקודה פסיק הוא ימשיך להוריד אותם:

```
?- retract(parent(kim,X)).
X = holly ;
X = kent.
```

אם נרצה להוריד את כולם בבת אחת נעשה retractall.